

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Y Greeshma Reddy	Reg. No.:	192424146
Section / Batch:	B Tech-AI&DS	Date:	13-08-2026

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y Greeshma Reddy

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.
2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.
3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.
4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.
5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. CFG Trees vs Dependency Parsing

Scope of Items

- Context-Free Grammar (CFG) represents a sentence as a hierarchical tree of constituents such as Sentence (S), Noun Phrase (NP), and Verb Phrase (VP).
- Dependency Parsing represents a sentence through direct relationships between words, where one word acts as the head and other words depend on it.
- CFG mainly focuses on phrase structure, whereas dependency parsing focuses on grammatical relationships between individual words.
- Both approaches are useful for syntactic analysis, but they provide different views of sentence structure.

Comparison Depth

- CFG is effective for identifying the internal structure of phrases and checking whether a sentence follows grammatical production rules.
- Dependency parsing is more direct for identifying relationships such as subject, object, modifier, and auxiliary.
- CFG can produce detailed hierarchical structures, but the relationships between distant words may not be immediately visible.
- Dependency parsing provides a simpler representation because each word is connected directly to its syntactic head.
- Therefore, dependency parsing is generally more suitable when the main requirement is to capture relationships between words.

Use of Evidence

- Consider the sentence: “The boy eats an apple.”
- CFG can represent it as $S \rightarrow NP + VP$, with the noun phrase “The boy” and verb phrase “eats an apple.”
- Dependency parsing can directly show:
 - eats $\rightarrow$ boy as subject
 - eats $\rightarrow$ apple as object
 - apple $\rightarrow$ an as determiner
- This clearly demonstrates how dependency parsing identifies grammatical relationships.

Critical Thinking

- CFG is preferable when the system needs detailed phrase-level information, such as identifying noun and verb phrases.
- Dependency parsing is more effective for applications involving semantic analysis, information extraction, question answering, and relation identification.

- Since the question specifically emphasizes relationships between words, dependency parsing provides a more useful representation.
- Hence, dependency parsing is the better choice for capturing direct grammatical relationships between words.

Clarity of Presentation

- CFG → focuses on hierarchical constituent structure.
- Dependency Parsing → focuses on head-dependent word relationships.
- CFG is useful for phrase-based grammar analysis, while dependency parsing is better for word-level relationships.

Conclusion: Dependency parsing is more effective when the objective is to clearly capture relationships between words.

Q2. Top-Down Parsing vs Earley Parsing

Scope of Items

- Top-Down Parsing begins with the start symbol of the grammar and attempts to generate the input sentence by applying grammar rules.
- It moves from the general sentence structure toward individual words.
- Earley Parsing is a dynamic-programming parsing method that maintains a chart of possible parsing states.
- Earley parsing is capable of handling ambiguous, incomplete, and complex sentences.

Comparison Depth

- Top-down parsing can be simple and efficient for small, well-defined grammars.
- However, when a sentence is ambiguous, it may need to explore several possible grammar rules, leading to backtracking and increased processing time.
- Earley parsing stores intermediate results, so the same partial structures do not need to be calculated repeatedly.
- It can handle left-recursive and ambiguous CFGs, making it more flexible than basic top-down parsing.
- For dynamic input, Earley parsing can maintain several possible interpretations until enough information is available.

Use of Evidence

- Consider an incomplete input: “The student wants to...”
- A top-down parser may have difficulty deciding which grammatical structure should be followed when the sentence is unfinished.
- Earley parsing can maintain multiple possible states and continue parsing as additional words

arrive.

- For an ambiguous sentence, Earley parsing can also retain multiple possible interpretations instead of immediately rejecting one.

Critical Thinking

- Top-down parsing is useful when the grammar is simple and the input is predictable.
- However, real-time systems often receive incomplete, noisy, or ambiguous user input.
- Earley parsing provides greater flexibility because it can process general CFGs and preserve alternative interpretations.
- Although Earley parsing may require more memory, its flexibility makes it more suitable for dynamic NLP applications.
- Therefore, Earley parsing is the better choice for applications dealing with incomplete and ambiguous user input.

Clarity of Presentation

- Top-Down Parsing → simple, predictive, but may suffer from backtracking.
- Earley Parsing → flexible, dynamic, and supports ambiguity.

Conclusion: Earley parsing is more suitable when input is incomplete or ambiguous and the system needs robust parsing.

3. CFG vs PCFG vs Neural Parsing

Scope of Items

- CFG describes the possible syntactic structures of a sentence using grammar rules.
- PCFG (Probabilistic CFG) extends CFG by assigning probabilities to grammar rules.
- Neural Parsing uses machine-learning and neural-network models to learn syntactic patterns from large amounts of language data.
- These approaches differ mainly in how they identify and resolve multiple possible interpretations.

Comparison Depth

- CFG can identify all grammatically valid interpretations but does not provide a mechanism for deciding which interpretation is more likely.
- PCFG assigns probabilities to alternative structures and can therefore select the most probable parse.
- Neural parsers can use contextual information, word representations, and patterns learned from large datasets.
- PCFG depends strongly on the quality of its grammar and probability estimates.
- Neural parsing can capture more complex linguistic patterns and contextual dependencies than traditional rule-based approaches.

Use of Evidence

- Consider: “She saw the man with a telescope.”
- This sentence has at least two interpretations:
 - She used a telescope to see the man.
 - The man had a telescope.
- CFG can represent both possible structures but cannot determine which one is intended.
- PCFG can assign probabilities to both structures and select the more probable interpretation.
- A neural parser can use contextual information from surrounding sentences to determine the intended meaning more effectively.

Critical Thinking

- CFG is transparent and easy to understand, but its ability to resolve ambiguity is limited.
- PCFG improves ambiguity handling because it ranks different grammatical structures using probabilities.
- Neural parsing generally provides better performance in real-world applications because it can learn complex patterns from large datasets and use contextual information.
- However, neural models require large training datasets, computational resources, and appropriate training.
- Therefore, neural parsing is generally the most effective for real-world ambiguity resolution, while PCFG remains useful when interpretability and probabilistic grammar rules are important.

Clarity of Presentation

- CFG → identifies possible grammatical structures.
- PCFG → identifies and ranks structures using probabilities.
- Neural Parsing → learns contextual patterns and resolves ambiguity using trained models.

Conclusion: Neural parsing is the most suitable for modern real-world applications involving complex and ambiguous language.

Q4. Feature Structures vs Subcategorization Frames

Scope of Items

- Feature Structures represent grammatical properties such as number, person, tense, gender, case, and agreement.
- Subcategorization Frames describe the types and number of arguments that a verb requires.
- Feature structures mainly help maintain grammatical feature compatibility, while subcategorization frames focus on verb argument requirements.
- Both are important for developing grammatically accurate NLP systems.

Comparison Depth

- Feature structures can ensure that the subject and verb agree in features such as number and person.
- For example, “She eats” is correct, while “She eat” violates subject–verb agreement.
- Subcategorization frames specify whether a verb requires a direct object, indirect object, complement, or other argument.
- For example, the verb “give” commonly requires a subject, an indirect object, and a direct object: “She gave him a book.”
- Therefore, feature structures and subcategorization frames solve different but complementary grammatical problems.

Use of Evidence

- Sentence: “He play football.”
 - The feature structure identifies a mismatch between singular subject “He” and the verb form “play.”
 - The correct form is “He plays football.”
- Sentence: “She gave.”
 - A subcategorization frame can identify that the verb “give” normally requires additional arguments.
 - “She gave him a book” satisfies the required argument structure.

Critical Thinking

- Feature structures are stronger for enforcing subject–verb agreement and other grammatical feature constraints.
- Subcategorization frames are better for ensuring that verbs receive the correct number and type of arguments.
- Since the system needs to handle both agreement and argument structure, using both approaches together would provide the strongest grammatical analysis.
- If one must be selected for overall grammatical constraint checking, feature structures provide broader support for agreement and feature compatibility, while subcategorization frames provide specialized support for verb argument structures.
- Therefore, a combined approach is ideal, with feature structures enforcing agreement and subcategorization frames enforcing verb argument requirements.

Clarity of Presentation

- Feature Structures → agreement and grammatical features.
- Subcategorization Frames → verb argument requirements.
- Feature structures are particularly useful for subject–verb agreement.
- Subcategorization frames are essential for checking whether verbs have appropriate complements.

Conclusion: Combining both provides the most accurate grammatical analysis.

5. Transition-Based vs Graph-Based Dependency Parsing

Scope of Items

- Transition-Based Parsing constructs a dependency tree step-by-step using a sequence of parsing actions such as shift, reduce, and attach.
- Graph-Based Parsing treats parsing as a global optimization problem and selects the dependency tree with the highest overall score.
- Transition-based parsing focuses mainly on local decisions, while graph-based parsing considers global relationships.
- Both methods are widely used for dependency parsing.

Comparison Depth

- Transition-based parsers make one decision at a time and use the current parser state to determine the next action.
- Because decisions are made sequentially, transition-based parsing is generally fast and computationally efficient.
- Graph-based parsers consider many possible relationships between words and select the best complete dependency structure.
- This global approach can improve structural consistency but may require more computational resources.
- Transition-based methods are therefore often preferred when processing speed and scalability are important.

Use of Evidence

- Suppose a large NLP system must parse millions of sentences.
- A transition-based parser can process each sentence through a sequence of efficient actions, making it suitable for high-throughput processing.
- A graph-based parser evaluates possible dependency connections and searches for the best overall tree.
- Graph-based parsing may provide strong global decisions, but the additional computation can become expensive on very large datasets.

Critical Thinking

- Graph-based parsing is useful when global accuracy and complete structural consistency are more important than processing speed.
- Transition-based parsing is more suitable when the system must process a large amount of data quickly.
- Modern transition-based systems can also use machine-learning models to improve local decisions while maintaining high speed.

- For large-scale applications, processing millions of sentences efficiently is a major requirement.
- Therefore, transition-based dependency parsing is generally more suitable for large-scale NLP systems because of its speed, scalability, and efficient decision-making.

Clarity of Presentation

- Transition-Based → local decisions, sequential processing, faster execution.
- Graph-Based → global decisions, complete tree optimization, higher computational cost.
- Transition-based parsing is preferred when scalability and speed are critical.

Conclusion: Transition-based parsing is the better choice for fast and large-scale dependency parsing.